

Topic 8: Linked Lists (Singly Linked List, Operations, Edge Cases & Complexity)

1. High-Yield Conceptual Theory

1.1 Array vs. Linked List Architectural Comparison

Characteristic	Array	Singly Linked List (SLL)
Memory Allocation	Contiguous block (static or dynamic)	Non-contiguous nodes scattered across the heap
Access Time	$\mathcal{O}(1)$ random access via index	$\mathcal{O}(n)$ sequential access from head
Binary Search	$\mathcal{O}(\log n)$ possible if sorted	$\mathcal{O}(n)$ (cannot jump to middle in $\mathcal{O}(1)$)
Insertion/Deletion at Head	$\mathcal{O}(n)$ (requires shifting elements)	$\mathcal{O}(1)$ (pointer manipulation only)
Memory Overhead	Zero per-element pointer overhead	High: requires 1 address pointer per node
Cache Locality	Excellent (spatial locality)	Poor (cache misses due to scattered heap addresses)

1.2 Self-Referential Structure Definition in C

A linked list node is represented using a **self-referential structure**:

```
struct Node {
    int data;           // 4 bytes on 32/64-bit systems
    struct Node *next;  // 4 bytes (32-bit) or 8 bytes (64-bit)
};
```

- The member `next` is a pointer to another structure of the **same tag name**.
- The end of the list is marked by a sentinel `NULL` pointer:



1.3 Asymptotic Complexity of Primitive Operations

Let n be the number of nodes in the list.

Operation	Time Complexity (Head only)	Time Complexity (Head + Tail pointer)	Space Complexity
Traverse / Search / Length	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Insert at Beginning (Head)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Insert at End (Tail)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Insert After Node `p`	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Insert Before Node `p`	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Delete from Head	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Delete from Tail	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Delete Node <code>p</code> (only <code>p</code> given)	$\mathcal{O}(1)$ (if <code>p</code> tail)	$\mathcal{O}(1)$ (if <code>p</code> tail)	$\mathcal{O}(1)$

Critical GATE Observation: Even with a dedicated `tail` pointer, **deleting the last node in a Singly Linked List requires $\mathcal{O}(n)$ time**. To update the tail pointer and free the last node, you must find the $(n - 1)$ -th node (the node right before the tail), which requires traversing all the way from `head`.

1.4 Detailed Pointer Manipulation Mechanics

1. Insertion After Node `curr`

```
struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
newNode->data = val;

// Order of execution is CRITICAL:
newNode->next = curr->next; // Step 1: Link new node to downstream chain
curr->next = newNode;       // Step 2: Link current node to new node
```

- **Order Trap:** Swapping these two statements (`curr->next = newNode; newNode->next = curr->next;`) causes a **self-loop** (`newNode->next = newNode`) and detaches the entire downstream list, causing a massive memory leak!

2. Deletion of Node Given Only a Pointer to That Node (The $\mathcal{O}(1)$ Trick)

If you are given a pointer `p` to a node inside a singly linked list (without access to `head`), and `p` is **not the last node**:

```
void deleteNodeWithoutHead(struct Node *p) {
    struct Node *temp = p->next;
    p->data = temp->data;      // Copy data from next node
    p->next = temp->next;     // Bypass next node
    free(temp);              // Free redundant node
}
```

- **Limitation:** This trick **fails if `p` is the last node** (`p->next == NULL`), because there is no successor node whose data can be copied. Deleting the last node strictly requires traversing from `head` to update the $(n - 1)$ -th node's pointer to `NULL`.

1.5 Two-Pointer Algorithms (Tortoise and Hare)

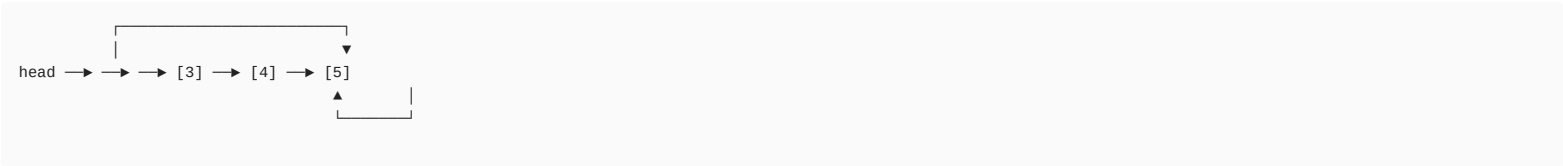
1. Finding the Middle Node

Let `slow` advance 1 step and `fast` advance 2 steps per iteration:

```
struct Node *slow = head, *fast = head;
while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
}
// 'slow' now points to the middle node
```

- **Odd length ($2k + 1$ nodes):** Loop terminates when `fast->next == NULL`. `slow` points exactly to the $(k + 1)$ -th node (exact middle).
- **Even length ($2k$ nodes):** Loop terminates when `fast == NULL`. `slow` points to the $(k + 1)$ -th node (the second middle node).

2. Floyd's Cycle Detection & Cycle Start Point



1. **Detection:** Initialize `slow = head`, `fast = head`. If a cycle exists, `fast` and `slow` will meet at some node inside the loop in $\mathcal{O}(n)$ time.
2. **Finding Cycle Start:**

- Let L be the distance from `head` to the loop entrance.
- Let m be the circumference of the loop.
- Let d be the distance from the loop entrance to the meeting point.
- When they meet:

$$\text{distance traveled by slow} = L + d$$

$$\text{distance traveled by fast} = L + d + m \quad (m \geq 1)$$

- Since `fast` moves twice as fast as `slow`:

$$2(L + d) = L + d + m \implies L + d = m \implies L = m - d = (m - 1) + (-d)$$

- **Algorithm:** Keep `fast` at the meeting point. Reset `slow = head`. Move both `slow` and `fast` at **1 step per iteration**. They will meet precisely at the **cycle start node**.

1.6 Memory Overhead Formulation

For an array of n elements vs. an SLL of n elements:

- Data payload per element = D bytes.
- Pointer address width = A bytes (4 bytes on 32-bit, 8 bytes on 64-bit).
- Struct padding/alignment = A bytes.

$$\text{memory}_{\text{Array}} = n \times D$$

$$\text{memory} = n \times (D + A)$$

$$\text{relative storage overhead} = \frac{n(D + A) - nD}{nD} = \frac{A}{D}$$

- When D is small (e.g., `char` or `int`), the linked list storage overhead can be 100 to 300 higher than a flat array.

2. Common GATE Pitfalls & Edge Cases

1. Lost Head Pointer during Traversal

```
// INCORRECT:
while (head != NULL) {
    printf("%d ", head->data);
    head = head->next; // BUG: Destroys original head pointer! List is permanently lost.
}
```

```
// CORRECT:
struct Node *curr = head;
while (curr != NULL) {
    printf("%d ", curr->data);
    curr = curr->next;
}
```

2. Null Pointer Dereferencing on Edge Boundaries

In fast-pointer checks:

```
while (fast->next != NULL && fast != NULL) // RUNTIME ERROR!
```

- If `fast == NULL`, evaluating `fast->next` produces a **Segmentation Fault**.
- Due to short-circuit evaluation, it **must** be written as:

```
while (fast != NULL && fast->next != NULL)
```

3. Memory Leak on Node Deletion

```
// LEAKY:
head = head->next; // The original head node is orphaned in the heap!

// SAFE:
struct Node *temp = head;
head = head->next;
free(temp);
```

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2010)

Question:

The following C function takes a singly linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers (1, 2, 3, 4, 5, 6, 7) in that given order.

```
void rearrange(struct Node *list) {
    struct Node *p, *q;
    int temp;
    if (!list || !list->next) return;
    p = list;
    q = list->next;
    while (q) {
        temp = p->data;
        p->data = q->data;
        q->data = temp;
        p = q->next;
        q = p ? p->next : 0;
    }
}
```

What is the resulting list after the function executes?

Step-by-step Solution:

Trace the pointer movements and data swaps:

1. Initial list: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7.
2. Initial pointers: p points to node 1, q points to node 2.
3. **Iteration 1:**

- Swap `p->data` and `q->data`: Swaps values 1 and 2.
- List becomes: 2 -> 1 -> 3 -> 4 -> 5 -> 6 -> 7.
- Update pointers:
- `p = q->next` → p points to node 3.
- `q = p ? p->next : 0` → q points to node 4.

4. Iteration 2:

- Swap `p->data` and `q->data`: Swaps values 3 and 4.
- List becomes: 2 -> 1 -> 4 -> 3 -> 5 -> 6 -> 7.
- Update pointers:
- `p = q->next` → p points to node 5.
- `q = p ? p->next : 0` → q points to node 6.

5. Iteration 3:

- Swap `p->data` and `q->data`: Swaps values 5 and 6.

- List becomes: 2 -> 1 -> 4 -> 3 -> 6 -> 5 -> 7 .
- Update pointers:
- $p = q \rightarrow \text{next} \rightarrow p$ points to node 7 .
- $q = p ? p \rightarrow \text{next} : 0 \rightarrow p \rightarrow \text{next}$ is NULL , so $q = \text{NULL}$.

6. Termination:

- while (q) evaluates to false.

7. Final list: 2, 1, 4, 3, 6, 5, 7 .

Output: 2, 1, 4, 3, 6, 5, 7

PYQ 2 (GATE CS 2004)

Question:

Consider the following C function that takes a reference to the head of a singly linked list and an integer k :

```
void mystery(struct Node **head, int k) {
    struct Node *p = *head, *q = NULL;
    int count = 0;
    while (p != NULL && count < k) {
        q = p;
        p = p->next;
        count++;
    }
    if (p == NULL) return;
    q->next = NULL;
    struct Node *temp = p;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = *head;
    *head = p;
}
```

What operation does this function perform on a list of n elements (where $k < n$)?

Step-by-step Solution:

1. Let the list be: $N_1 \rightarrow N_2 \rightarrow \dots \rightarrow N_k \rightarrow N_{k+1} \rightarrow \dots \rightarrow N_n \rightarrow$.
2. The first loop runs until `count == k` :

- q stops at node N_k .
- p stops at node N_{k+1} .

3. Disconnection:

- $q \rightarrow \text{next} = \text{NULL}$; cuts the list into two parts:
- Part 1: $N_1 \rightarrow N_2 \rightarrow \dots \rightarrow N_k \rightarrow$.
- Part 2: $N_{k+1} \rightarrow N_{k+2} \rightarrow \dots \rightarrow N_n \rightarrow$ (pointed to by p).

4. Reconnection:

- temp traverses Part 2 to reach its tail N_n .
- $\text{temp} \rightarrow \text{next} = *head$; attaches Part 1 to the end of Part 2:

$$N_{k+1} \rightarrow N_{k+2} \rightarrow \dots \rightarrow N_n \rightarrow N_1 \rightarrow \dots \rightarrow N_k \rightarrow$$

- $*head = p$; updates head to point to N_{k+1} .

5. This is a **left cyclic rotation** of the list by k positions (or rotating by k nodes).

Answer: Left cyclic rotation of the linked list by k nodes.

PYQ 3 (GATE CS 2011)

Question:

Suppose you are given a pointer p to an arbitrary node in a singly linked list (not necessarily the head). You are **not** given the pointer to the head of the list. Under what condition can you delete the node pointed to by p in $\mathcal{O}(1)$ time?

(A) Under all conditions

(B) If and only if p is not the first node

(C) If and only if p is not the last node

(D) It is impossible to delete p without access to the head

Step-by-step Solution:

1. To delete a node in a singly linked list normally, its predecessor must point to its successor.
2. Without `head`, you cannot locate the predecessor of p in $\mathcal{O}(1)$ time.
3. However, if p is **not the last node** ($p \rightarrow \text{net}$):

- Copy the data of $p \rightarrow \text{net}$ into p .
 - Set $p \rightarrow \text{net} = p \rightarrow \text{net} \rightarrow \text{net}$.
 - Free the successor node.
 - This effectively removes the node's original data in $\mathcal{O}(1)$ time.
4. If p is the last node, this trick cannot be used because there is no successor node to copy data from.
5. Therefore, it is possible if and only if p is not the last node.

Correct Option: (C)

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Finding k -th Node from End in Single Pass

Question:
To find the k -th node from the end of a singly linked list containing n nodes ($k \leq n$) in a **single traversal**, how many total node visits are performed?

```
struct Node *findKthFromEnd(struct Node *head, int k) {
    struct Node *p1 = head, *p2 = head;
    for (int i = 0; i < k; i++) {
        if (p1 == NULL) return NULL;
        p1 = p1->next;
    }
    while (p1 != NULL) {
        p1 = p1->next;
        p2 = p2->next;
    }
    return p2;
}
```

What is the exact number of pointer advances performed?

Detailed Explanation:

1. Phase 1: Pointer $p1$ is advanced k times.
 - $p1$ is now at distance k from head (at the $(k + 1)$ -th node from the beginning).
2. Phase 2: Both $p1$ and $p2$ move simultaneously 1 step at a time until $p1$ becomes NULL .
 - Distance from the $(k + 1)$ -th node to NULL is $n - k$ steps.
 - Therefore, the `while` loop runs exactly $n - k$ iterations.
3. Total advances made by $p1$: $k + (n - k) = n$ steps.
4. Total advances made by $p2$: $n - k$ steps.
5. Combined pointer steps = $n + (n - k) = 2n - k$.
6. Time Complexity is strictly $\mathcal{O}(n)$, completed in a single pass of $p1$.

Output: $2n - k$ steps, $\mathcal{O}(n)$ Time, $\mathcal{O}(1)$ Space

Practice Question 2: Floyd's Algorithm Meeting Point

Question:
A linked list contains a total of $N = 100$ nodes. The first 40 nodes form a linear chain ($1 \rightarrow 2 \rightarrow \dots \rightarrow 40$), and node 40 connects to node 41. Node 100 connects back to node 41, forming a cycle of length $= 60$.
If `slow` and `fast` pointers both start at node 1 (head), at which specific node number will they meet for the first time?

Detailed Explanation:

1. Let node numbers be indexed by distance from head:
 - Node 1 is at distance 0.
 - Node 41 (cycle start) is at distance $L = 40$.
 - Loop size $= 60$.
2. When `slow` reaches the start of the loop (node 41):
 - Time elapsed $t = 40$ steps.
 - Distance moved by `slow` = 40.
 - Distance moved by `fast` = $2 \times 40 = 80$.
3. Position of `fast` inside the cycle at $t = 40$:
 - `fast` has traveled $80 - 40 = 40$ steps into the cycle.
 - Relative distance behind `slow` in the cycle:

$$\text{ap etween slow and fast} = (60 - 40) = 20 \text{ steps}$$
4. Relative speed between `fast` and `slow`:

$$\text{elatie speed} = 2 - 1 = 1 \text{ step per time nit}$$
5. Time needed for `fast` to catch up to `slow`:

$$t = \frac{\text{distance}}{\text{relative speed}} = \frac{40 \pmod{60}}{1} = 20 \text{ steps}$$

(Since fast gains 1 step per cycle iteration, it takes $60 - 20 = 40$ steps to close a 40-step gap, meaning 40 steps).

Let us verify formally:

$$\text{ist}_{\text{slow}}(t) = 40 + k$$

$$\text{ist}_{\text{fast}}(t) = 80 + 2k$$

$$\text{ollision ondition } (80 + 2k) - (40 + k) \equiv 0 \pmod{60}$$

$$40 + k \equiv 0 \pmod{60} \implies k = 20$$

6. After $k = 20$ steps from the entrance:

- Position of `slow` = `ntrane node` + 20 = 41 + 20 = 61.

7. They meet at node **61**.

Output: Node `61`

Practice Question 3: Pointer Assignment Swapping Bug

Question:

Consider the following attempt to insert a new node `newNode` before node `curr`:

```
newNode->next = curr;
prev->next = newNode;
```

If `curr == head`, what additional condition check is required to prevent program failure?

Detailed Explanation:

- If `curr == head`, there is no `prev` node (`prev == NULL`).
- The statement `prev->next = newNode;` will attempt to dereference a `NULL` pointer (`NULL->next`), triggering an immediate **Segmentation Fault**.
- To handle insertion at head:

```
if (curr == head) {
    newNode->next = head;
    head = newNode;
} else {
    newNode->next = curr;
    prev->next = newNode;
}
```

5. Quick Revision & Edge-Case Checklist

- ☐ SLL cannot support Binary Search efficiently; access is strictly sequential ($\mathcal{O}(n)$).
- ☐ Deleting from the tail in an SLL is always $\mathcal{O}(n)$, even if a `tail` pointer is maintained.
- ☐ Insertion after node `p`: `newNode->next = p->next`; `p->next = newNode`; . Reversing these lines creates an infinite self-loop.
- ☐ Deleting node `p` without `head` in $\mathcal{O}(1)$ is only possible if *p* is **not the last node**.
- ☐ In fast/slow pointers, always verify `fast != NULL && fast->next != NULL` before advancing `fast->next->next`.
- ☐ To find cycle start in Floyd's algorithm: after meeting, reset one pointer to `head`, move both at 1 step/iteration; they meet at the cycle start.
- ☐ SLL memory overhead = $n \times \text{sizeof(pointer)}$ bytes compared to arrays.

This concludes **Topic 8: Linked Lists (Singly Linked List)**. Reply with **"Continue"** to proceed to the next topic: **Doubly Linked List (DLL)**.